IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Buscar buenos hiperparámetros

Teoría, práctica y, al final, material de consulta

Sprint 8 · S08_05 · teoría breve, 40 min de práctica en casa y material de consulta

CONCEPTOS

Parámetros frente a hiperparámetros

	Parámetros	Hiperparámetros
Quién los fija	El **modelo**, al entrenar	**Nosotros**, antes de entrenar
Ejemplos	El umbral que elige un árbol	**max_depth**, **C**, **learning_rate**, **k**

Con 8 reservas (5 a 150 días de antelación) fijo yo **max_depth=2**; el corte en **50 días** lo aprende el árbol.

Los hiperparámetros son los botones del horno: los giras tú antes de cocinar. Los parámetros salen de la cocción.

QUÉ HACE UN BUSCADOR

Un buscador gira los botones por ti

1 · Elige

Una combinación de hiperparámetros.

2 · Evalúa

Con **validación cruzada**, solo sobre train.

3 · Repite

Con todas las combinaciones que le des.

4 · Reentrena

refit=True: la mejor, con todo train.

Rejilla	Combinaciones	Pliegues	Entrenamientos
max_depth [2, 4, 6] × **min_samples_leaf** [1, 5] × **criterion** [gini, entropy]	3 × 2 × 2 = 12	4	12 × 4 = 48, **+ 1** del reentrenado = 49

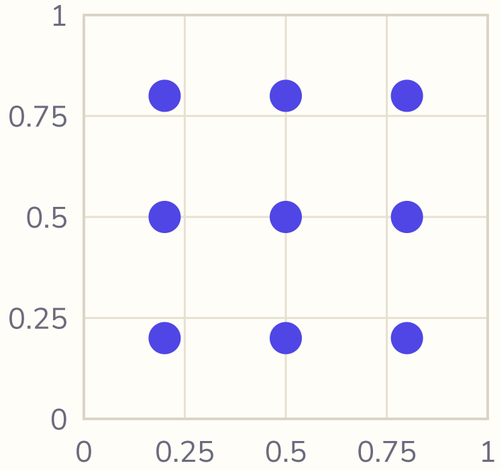
Cada hiperparámetro nuevo multiplica el coste: por eso la rejilla se dispara.

REJILLA FRENTE A AZAR

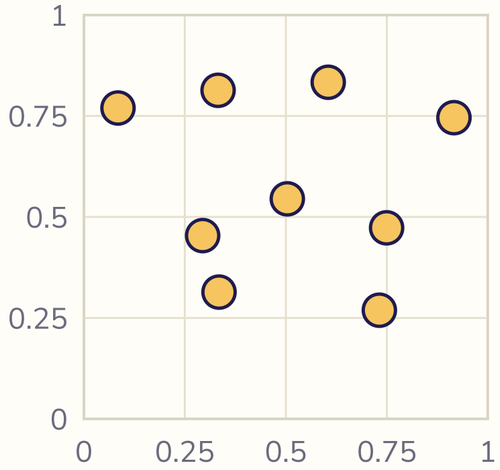
Con 9 pruebas, el azar explora más

Eje horizontal: un hiperparámetro que **importa**. Eje vertical: otro que **apenas influye**.

Rejilla · 3 valores



Al azar · 9 valores



Rejilla

9 pruebas, pero solo **3 valores distintos** del hiperparámetro que importa.

Aleatorio

9 pruebas y **9 valores distintos** del que importa, con el mismo coste.

RandomizedSearchCV prueba n_iter combinaciones al azar: explora más con el mismo presupuesto.

BÚSQUEDA ALEATORIA

Distribuciones en lugar de listas

Distribución	Qué genera	5 valores de ejemplo
randint(2, 12)	Enteros del 2 al **11** (el 12 no entra)	7 · 10 · 11 · 7 · 2
loguniform(0.001, 1)	Decimales repartidos por **órdenes de magnitud**	0.0178 · 0.1449 · 0.0010 · 0.0081 · 0.0028
uniform(0.5, 0.5)	Decimales de 0,5 a 1,0: **(inicio, ancho)**	0.7085 · 0.8602 · 0.5001 · 0.6512 · 0.5734

Por qué loguniform

Un tercio de las pruebas cae por debajo de 0,01 y otro tercio por encima de 0,1. Con una uniforme, **solo el 1 %** bajaría de 0,01.

n_iter fija el presupuesto

n_iter=25 combinaciones × 3 pliegues = **75** entrenamientos: eliges tú cuánto gastar.

uniform(inicio, ancho): **no** es (mínimo, máximo).

LEER LOS RESULTADOS

Muchos empates: elige lo sencillo

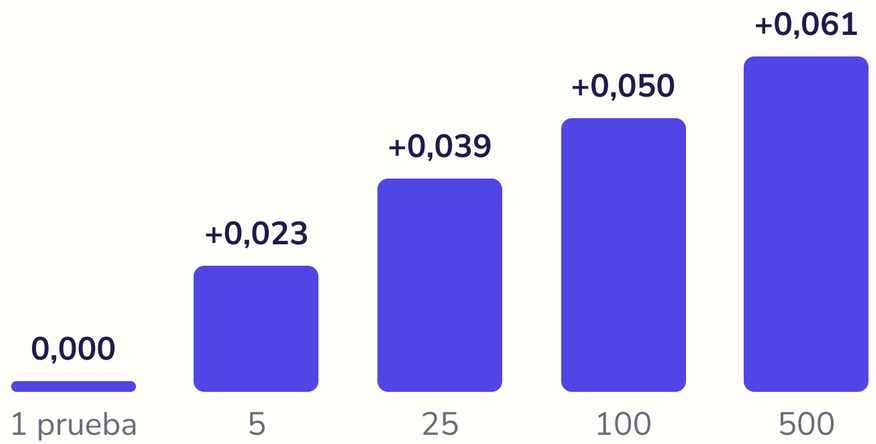
max_depth	min_samples_leaf	mean_test_score	std_test_score	¿Dentro del umbral?
10	21	0.892	0.022	Sí
6	13	0.886	0.027	Sí
6	19	0.884	0.024	Sí
9	10	0.874	0.035	Sí
8	7	0.866	0.036	No
12	6	0.859	0.022	No

`pd.DataFrame(busqueda.cv_results_)`: 8 combinaciones, las 6 mejores. Umbral 0,892 − 0,022 = **0,870**: dentro, **4 de 8**.

Entre las empatadas, la más sencilla: `max_depth` = 6, no 10. Mismo rendimiento, menos riesgo.

LA NOTA OPTIMISTA

La mejor nota de validación es en parte suerte



Ejemplo con el árbol

Validación **0.892** → test **0.850**: 0.042 menos.

Qué lo explica

Entre muchas combinaciones, alguna sale bien por azar: la «maldición del ganador».

El test se usa una sola vez, al final, con el modelo ya elegido. Nunca para elegir.

PRÁCTICA · CUADERNO S08_05 (CASA)

Lo que haremos en el cuaderno

Parte	Qué haremos	Dónde
Preparación	Preparar las herramientas y los datos	Casa
Ejemplo	Por qué el aleatorio explora mejor que la rejilla	Casa
Ejercicio 1	GridSearchCV con un bosque (necesario para el reto)	Casa
Ejercicio 2	RandomizedSearchCV con LightGBM (necesario para el reto)	Casa
Ejercicio 3	Configuraciones empatadas: la regla del 1 error estándar	Casa
Ejemplo	AUC según el learning_rate	Casa
Ejercicio 4	La nota de la búsqueda frente al test	Casa
Mini-reto	El informe de la búsqueda	Casa

Cada ejercicio: **lee** → **escribe** → **ejecuta** → **comprueba el ✔**. La solución está plegada: ábrela tras intentarlo.

CONSULTA

Material de consulta: código y aclaraciones

Cuándo abrirlo

Después de intentar el ejercicio, o cuando algo no te salga. En clase no se proyecta.

Cómo está ordenado

En el **mismo orden que el cuaderno**: ejercicios 1 a 4 y mini-reto. Cada uno trae su código y sus aclaraciones.

Ejercicio	Tema	Diapositivas
1	GridSearchCV con un bosque	10 y 11
2	RandomizedSearchCV con LightGBM	12 y 13
3	Empates: 1 error estándar	14 y 15
4	Nota de búsqueda y test	16 y 17
Mini-reto	Informe de la búsqueda	18 y 19

CONSULTA · EJERCICIO 1

GridSearchCV con un bosque (ejercicio 1)

```
pipe_rf = Pipeline([("prep", prep), ("rf", RandomForestClassifier(random_state=SEMILLA, n_jobs=2))])
rejilla = {"rf__n_estimators": [100, 200], "rf__max_depth": [4, 8], "rf__min_samples_leaf": [10, 40]}
busqueda_grid = GridSearchCV(pipe_rf, rejilla, cv=cv, scoring="roc_auc").fit(X_train, y_train)
n_combinaciones = len(busqueda_grid.cv_results_["params"])    # 8
mejor_auc_grid = busqueda_grid.best_score_                    # 0.747
```

8 → 24 entrenamientos

combinaciones × 3 pliegues

0,747

AUC en validación cruzada

Mejores: **max_depth = 8, min_samples_leaf = 40, n_estimators = 100.**

CONSULTA · ACLARACIONES · EJERCICIO 1

Leer la rejilla con el bosque (ejercicio 1)

rf__max_depth, no max_depth

max_depth es de **el paso rf** del Pipeline: la forma es paso__parametro, con dos guiones bajos.

GridSearchCV(..., cv=cv)

Prueba **todas** las combinaciones con validación cruzada, solo con X_train: el test no interviene.

2 × 2 × 2 = 8

8 combinaciones × 3 pliegues = **24 entrenamientos** (más el reentrenado final con todo train).

Si olvidas el prefijo

ValueError: Invalid parameter 'max_depth' for estimator Pipeline(...). En el reto el paso se llama **clf**: allí es **clf__max_depth**.

CONSULTA · EJERCICIO 2

RandomizedSearchCV con LightGBM (ejercicio 2)

```
pipe_lgb = Pipeline([("prep", prep), ("lgb", lgb.LGBMClassifier(n_estimators=200, subsample_freq=1, random_state=SEMILLA, n_jobs=2,
    verbose=-1))])
distribuciones = {"lgb__learning_rate": loguniform(0.01, 0.3), "lgb__num_leaves": randint(3, 40),
                  "lgb__min_child_samples": randint(10, 200), "lgb__subsample": uniform(0.6, 0.4)}
busqueda_rand = RandomizedSearchCV(pipe_lgb, distribuciones, n_iter=25, cv=cv, scoring="roc_auc",
                                   random_state=SEMILLA).fit(X_train, y_train)
mejor_auc_rand = busqueda_rand.best_score_    # 0.749
```

25 → 75 entrenamientos

combinaciones × 3 pliegues

0,749

AUC en validación cruzada

Mejor: **learning_rate** ≈ 0,019, **num_leaves** = 23, **min_child_samples** = 199, **subsample** ≈ 0,755.

CONSULTA · ACLARACIONES · EJERCICIO 2

Leer la búsqueda aleatoria (ejercicio 2)

loguniform(0.01, 0.3)

Reparte los valores por **órdenes de magnitud**: tan a menudo 0,01–0,03 como 0,1–0,3. Con una uniforme casi todos serían mayores de 0,1.

uniform(0.6, 0.4)

Es **(inicio, ancho)**: valores de 0,6 a 1,0. Y **subsample_freq=1** en el modelo, o **subsample** no tiene efecto.

n_iter=25

25 combinaciones al azar × 3 pliegues = **75 entrenamientos**: el presupuesto lo fijas tú. En el reto, máximo 5 minutos por búsqueda.

Prefijo lgb__

Como en la rejilla, cada nombre lleva el del paso. Sin **.fit(X_train, y_train)** no existe **best_score_** (**AttributeError**).

CONSULTA · EJERCICIO 3

Empates: la regla del 1 error estándar (ejercicio 3)

```
resultados_ord = pd.DataFrame(busqueda_rand.cv_results_).sort_values("mean_test_score", ascending=False)
mejor_fila = resultados_ord.iloc[0]
umbral_1se = mejor_fila["mean_test_score"] - mejor_fila["std_test_score"]
empatadas = resultados_ord[resultados_ord["mean_test_score"] >= umbral_1se]
n_empatadas = len(empatadas)                                          # 13
mas_simple = int(empatadas["param_lgb__num_leaves"].astype(int).min())    # 4
```

13 de 25

empatadas con la mejor

4 (frente a 23)

hojas de la más sencilla

Las cinco primeras: 0,749 · 0,749 · 0,749 · 0,748 · 0,747, con 23, 4, 18, 13 y 11 hojas.

CONSULTA · ACLARACIONES · EJERCICIO 3

Leer los empates y el learning_rate (ejercicio 3)

ascending=False

Sin él, la tabla queda de peor a mejor y **iloc[0]** es la **peor**. Después, **head(5)** da las cinco primeras.

umbral_1se

Media de la mejor **menos** su desviación: lo que quede por encima está **empatado** con ella.

23 hojas frente a 4

Rinden igual (0,749): la diferencia entre las cinco primeras (0,002) es **menor que la desviación** entre pliegues (≈ 0,012). Se elige la sencilla.

AUC según **learning_rate**

Con ≥ 0,15, el AUC baja (media 0,697, 4 combinaciones); con < 0,05 se mantiene (0,744, 12). **Importa el learning_rate**, no las hojas.

CONSULTA · EJERCICIO 4

La nota de la búsqueda frente al test (ejercicio 4)

```
auc_test_grid = roc_auc_score(y_test, busqueda_grid.best_estimator_.predict_proba(X_test)[:, 1])
auc_test_rand = roc_auc_score(y_test, busqueda_rand.best_estimator_.predict_proba(X_test)[:, 1])
optimismo_rand = mejor_auc_rand - auc_test_rand    # +0.009
```

0,747 → 0,734

rejilla + bosque: validación → test

0,749 → 0,740

aleatorio + LightGBM: validación → test

El optimismo es +0,009: la nota de la búsqueda fue algo más alta que la de test.

CONSULTA · ACLARACIONES · EJERCICIO 4

Leer la nota frente al test (ejercicio 4)

best_estimator_

Ya viene **reentrenado con todo train** (refit=True): se usa directamente para predecir sobre X_test.

predict_proba(X_test)[:, 1]

El AUC necesita **probabilidades**, no predict (0 o 1). Y es X_test, no X_train.

0,749 → 0,740

El optimismo es **+0,009**: es lo que tiene elegir la mejor de 25. Con la rejilla, 0,747 → 0,734.

Por qué es pequeño

Las configuraciones están **casi empatadas** (0,744–0,749): elegir entre ellas apenas hace trampa. Con configuraciones muy distintas sería mayor.

CONSULTA · MINI-RETO

El informe de la búsqueda (mini-reto)

```
informe = {"modelo": "lightgbm", "parametros": busqueda_rand.best_params_,
           "auc_validacion": round(mejor_auc_rand, 3), "auc_test": round(auc_test_rand, 3),
           "n_entrenamientos": 25 * 3,
           "recomendacion": "Test 0,740 frente a validación 0,749 y 13 empatadas: resultado fiable,
               mejorar más sería marginal."}
```

Clave	Valor
modelo	"lightgbm"
auc_validacion · auc_test · n_entrenamientos	0,749 · 0,74 · 75 (25 × 3)
recomendacion	frase que justifica si el resultado es fiable

Gana el aleatorio por AUC de validación (0,749 frente a 0,747), con 3 veces más entrenamientos.

CONSULTA · ACLARACIONES · MINI-RETO

Leer el informe (mini-reto, casa)

n_entrenamientos

Combinaciones × pliegues del buscador ganador: **75** (25 × 3) con el aleatorio, **24** con la rejilla. Sin contar el reentrenado.

recomendacion

Una **frase completa** que mencione la diferencia validación–test o los empates.

Qué ganó el aleatorio

Casi nada: **0,002** en validación (0,749 frente a 0,747) y 0,006 en test (0,740 frente a 0,734), con unas **3 veces** más entrenamientos.

¿Es fiable?

Sí: el test (0,740) es muy parecido a la validación (0,749) y hay **13 empatadas**. Mejorar más sería marginal.